

Interpréter l'AST SHDL.

Comment partir d'une chaîne SHDL, obtenir un arbre, l'inspecter visuellement, et parcourir les nœuds utiles pour construire le circuit.

1. Obtenir un `Module` pour travailler

Trois options selon le contexte. La plus rapide pour développer ton interprète est la première.

Option A — utiliser les exemples prêts à l'emploi (recommandé au début)

```
import parser.ll1.ast.Module;
import tests.parser.ll1.ast.ExemplesAst;

Module m = ExemplesAst.et();           // c = a * b
Module m = ExemplesAst.ou();           // c = /a + b
Module m = ExemplesAst.mux();          // c = s*a + /s*b
Module m = ExemplesAst.deuxSorties();  // x = a ; y = b
Module m = ExemplesAst.basculeD();     // hors scope MVP
Module m = ExemplesAst.decBCD();       // hors scope MVP (map)
```

Chaque méthode renvoie un `Module` déjà parsé, prêt à être passé à ton interprète. Ça évite tout contact avec le lexer ou les listes de tokens. Idéal tant que la config lexer SHDL d'Erwan n'est pas livrée.

Option B — parser une chaîne SHDL (quand le lexer sera prêt)

```
import parser.ll1.parser.Parser;
import parser.ll1.ast.Module;

Module m = Parser.parseFrom(source, lexer);
```

Option C — partir d'une liste de tokens

```
Module m = new Parser(tokens).parse();
```

Dans tous les cas le résultat est un `parser.ll1.ast.Module`. Toute erreur de syntaxe lève une `ParsingException` avec ligne, colonne et offset dans la source.

2. Visualiser l'arbre — `AstDumper.dump()`

Une méthode statique imprime l'arbre en texte indenté. Sans argument autre que la racine.

```
import parser.ll1.ast.AstDumper;
```

```
System.out.println(AstDumper.dump(module));
```

Les exemples ci-dessous sont des sorties réelles du dumper à gauche la source SHDL, à droite l'arbre produit.

Exemple 1 — porte ET

SHDL

```
module ET(a, b : c)
  c = a * b
end module
```

DUMP

```
Module "ET" [1.1]
├─ Params
│   ├── Signal "a"
│   ├── Signal "b"
│   └─ Signal "c"
└─ Instances
    └─ Assignment [1.1]
        ├── Target
        │   └─ SignalCompound
        │       └─ Signal "c"
        └─ Expr
            └─ SumOfTerms
                └─ Term
                    ├── Factor SIGNAL "a"
                    └─ Factor SIGNAL "b"
```

Exemple 2 — OU avec négation

SHDL

```
module OUN(a, b : c)
  c = /a + b
end module
```

DUMP

```
Module "OUN" [1.1]
├─ Params
│   ├── Signal "a"
│   ├── Signal "b"
│   └─ Signal "c"
└─ Instances
    └─ Assignment [1.1]
        ├── Target
        │   └─ SignalCompound
        │       └─ Signal "c"
        └─ Expr
            └─ SumOfTerms
                ├── Term
                │   ├── Factor NEG_SIGNAL /"a"
                └─ Term
                    └─ Factor SIGNAL "b"
```

Exemple 3 — multiplexeur $(s \cdot a) + (/s \cdot b)$

SHDL

```

module MUX(a, b, s : c)
  c = s*a + /s*b
end module

```

DUMP

```

Module "MUX" [l.1]
├─ Params
│  ├─ Signal "a"
│  ├─ Signal "b"
│  ├─ Signal "s"
│  └─ Signal "c"
└─ Instances
   └─ Assignment [l.1]
      ├─ Target
      │  └─ SignalCompound
      │     └─ Signal "c"
      └─ Expr
         └─ SumOfTerms
            ├─ Term
            │  ├─ Factor SIGNAL "s"
            │  └─ Factor SIGNAL "a"
            └─ Term
               ├─ Factor NEG_SIGNAL /"s"
               └─ Factor SIGNAL "b"

```

Exemple 4 — plusieurs sorties

SHDL

```

module DEUX(a, b : x, y)
  x = a
  y = b
end module

```

DUMP

```

Module "DEUX" [l.1]
├─ Params
│  ├─ Signal "a"
│  ├─ Signal "b"
│  ├─ Signal "x"
│  └─ Signal "y"
└─ Instances
   ├─ Assignment [l.1]
   │  ├─ Target
   │  │  └─ SignalCompound
   │  │     └─ Signal "x"
   │  └─ Expr
   │     └─ SumOfTerms
   │        └─ Term
   │           └─ Factor SIGNAL "a"
   └─ Assignment [l.1]
      ├─ Target
      │  └─ SignalCompound
      │     └─ Signal "y"
      └─ Expr
         └─ SumOfTerms
            └─ Term
               └─ Factor SIGNAL "b"

```

Exemple 5 — bascule D (hors scope MVP, pour info)

SHDL

```
module BasculeD(d, clk : q)
  q := d on clk, set when 1
end module
```

DUMP

```
Module "BasculeD" [l.1]
├ Params
│   ├── Signal "d"
│   ├── Signal "clk"
│   └── Signal "q"
├ Instances
│   └── MemoryPoint [SET] [l.1]
│       ├── Target
│       │   └── SignalCompound
│       │       └── Signal "q"
│       ├── Expr
│       │   └── SumOfTerms
│       │       └── Term
│       │           └── Factor SIGNAL "d"
│       ├── Clock
│       │   └── SumOfTerms
│       │       └── Term
│       │           └── Factor SIGNAL "clk"
│       └── Condition
│           └── SumOfTerms
│               └── Term
│                   └── Factor LITERAL_1 (= 1)
```

3. Les types à connaître

Tout est dans le package `parser.ll1.ast`. Seules les classes ci-dessous sont indispensables pour un interpréteur "équations booléennes combinatoires".

TYPE	À QUOI ÇA SERT	GETTERS UTILES
Module	Racine de l'arbre (un module SHDL).	<code>getName()</code> , <code>getParams()</code> , <code>getInstances()</code>
Signal	Identifiant de signal (peut être scalaire ou vecteur).	<code>getName()</code> , <code>getHi()</code> , <code>getLo()</code> (vecteur)
Instance	Interface : une ligne dans le corps du module.	implémentée par <code>Assignment</code> , <code>ModuleInstance</code> et les types hors scope.
Assignment	Une équation combinatoire <code>cible = expr</code> .	<code>getTarget()</code> , <code>getExprCompound()</code>
SignalCompound	Liste de signaux côte-à-côte (cible à gauche du <code>=</code>).	<code>getSignals()</code>
SumOfTerms	Somme (OU logique) de termes. Correspond à <code>a + b + c</code> .	<code>getTerms()</code>
Term	Produit (ET logique) de facteurs. Correspond à <code>a · b · c</code> .	<code>getFactors()</code>

TYPE	À QUOI ÇA SERT	GETTERS UTILES
Factor	Opérande élémentaire. Cinq variantes via <code>Kind</code> .	<code>getKind()</code> , <code>getSignal()</code> , <code>getInner()</code> , <code>getBitField()</code>

`Factor.Kind` est un `enum` à six valeurs :

- `SIGNAL` — référence à un signal, `getSignal()` donne le nom.
- `NEG_SIGNAL` — idem, mais nié (`/a`). À interpréter avec un `Not`.
- `LITERAL_0` — constante false. Brancher un `Constante(c, false)`.
- `LITERAL_1` — constante true. Brancher un `Constante(c, true)`.
- `BITFIELD` — littéral binaire multi-bits (`"1010"`). Hors scope MVP.
- `PAREN` — sous-expression entre parenthèses, `getInner()` donne le `SumOfTerms` interne.

4. Convention entrées / sorties

Le type `Signal` ne porte pas d'information de direction. Il faut la déduire du contexte. Deux règles simples :

- **Sortie** : tout signal qui apparaît comme `target` d'au moins un `Assignment` (ou autre `Instance`) dans le module. Concrètement : `assignment.getTarget().getSignals()`.
- **Entrée** : tout signal de `module.getParams()` qui n'est jamais une cible.

Alternative : dans la syntaxe SHDL l'en-tête `module M(e1, e2 : s1, s2)` met toujours les entrées avant le `:` et les sorties après. Le parser ne conserve pas cette frontière explicitement — les deux listes sont concaténées dans `getParams()`. La règle « scanner les `Assignment.getTarget()` » reste donc la plus fiable.

5. Squelette d'interprète — comment attaquer

Un visiteur parcourt l'arbre, alimente ton `DicoConnecteur` et instancie tes `Composant` au fur et à mesure. Voici la forme typique pour le cas combinatoire.

```
// Pour chaque Module :
DicoConnecteur dico = new DicoConnecteur();
for (Signal s : module.getParams()) {
    dico.getConnecteur(s.getName()); // crée un Lien si absent
}

// Pour chaque Assignment du corps :
for (Instance inst : module.getInstances()) {
    if (inst instanceof Assignment a) {
        Connecteur cible = dico.getConnecteur(
            a.getTarget().getSignals().get(0).getName());
        Connecteur resultat = interpreterExpr(a.getExprCompound().get(0), dico);
        // Brancher `resultat` sur `cible` (ou réutiliser directement).
    } else {
```

```

        // ModuleInstance, TriState, MemoryPoint, Fsm, MapNode -> hors scope MVP
        throw new UnsupportedOperationException(
            "Non supporté en MVP : " + inst.getClass().getSimpleName());
    }
}

```

Et pour l'expression booléenne :

```

Connecteur interpreterSomme(SumOfTerms s, DicoConnecteur d) {
    // un terme = ET de facteurs ; une somme = OU de termes.
    List<Connecteur> sorties = new ArrayList<>();
    for (Term t : s.getTerms()) sorties.add(interpreterProduit(t, d));
    return chaineOr(sorties); // crée un arbre de Or(...)
}

Connecteur interpreterProduit(Term t, DicoConnecteur d) {
    List<Connecteur> facteurs = new ArrayList<>();
    for (Factor f : t.getFactors()) facteurs.add(interpreterFacteur(f, d));
    return chaineAnd(facteurs);
}

Connecteur interpreterFacteur(Factor f, DicoConnecteur d) {
    return switch (f.getKind()) {
        case SIGNAL -> d.getConnecteur(f.getSignal().getName());
        case NEG_SIGNAL -> {
            Connecteur in = d.getConnecteur(f.getSignal().getName());
            Lien out = new Lien("neg_" + f.getSignal().getName());
            new Not(in, out);
            yield out;
        }
        case LITERAL_0 -> { Lien l = new Lien("_c0"); new Constante(l, false); yield l; }
        case LITERAL_1 -> { Lien l = new Lien("_c1"); new Constante(l, true); yield l; }
        case PAREN -> interpreterSomme(f.getInner(), d);
        case BITFIELD -> throw new UnsupportedOperationException("BITFIELD hors MVP");
    };
}

```

Les méthodes `chaineAnd(...)` / `chaineOr(...)` créent un arbre binaire de `And` / `Or` à partir d'une liste. Cas dégénéré (un seul élément) : retourner le connecteur tel quel.

6. Tests et outils disponibles

Le parser est couvert par ~114 tests JUnit (actuellement tous verts). Pertinents pour toi :

- `tests/parser/ll1/ast/ExemplesAst.java` — 6 méthodes statiques renvoyant des `Module` prêts à l'emploi (voir section 1). Ton point d'entrée principal pour développer et tester.
- `tests/parser/ll1/ast/ExemplesAstTest.java` — 6 tests qui vérifient la forme de chaque exemple. À lire si tu veux voir comment naviguer dans un `Module` depuis du code.

- `tests/parser/ll1/ast/AstDumperTest.java` — assertions sur des dumps d'arbre. À copier si tu veux figer le résultat de ton interprète sur un cas précis.
- `tests/parser/ll1/ast/AstDumperDemo.java` — harnais `main()` qui imprime les 6 dumps (5 montrés dans ce PDF + le décodeur BCD). Lance-le pour vérifier que tout compile chez toi.

Comment lancer

```
# Compile parser + tests
./build.sh test

# Lance toute la batterie (~25 suites)
./run.sh

# Lance une suite en particulier
java -cp "bin:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
    org.junit.runner.JUnitCore tests.parser.ll1.ast.AstDumperTest

# Lance le harnais d inspection
java -cp "bin:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
    tests.parser.ll1.ast.AstDumperDemo
```

Pour tes propres tests, crée `tests/simulateur/MonTest.java`, importe `ShdlFixtures` et `Parser`, et `./build.sh test` les récupère automatiquement — aucune config à toucher.

7. Ce qu'on ignore

Définitivement exclu du projet

Les machines à états finis (FSM) sortent totalement du périmètre de ce projet, à court comme à long terme. Le parser conserve les nœuds pour qu'un module avec `fsm ... end fsm` puisse tout de même être parsé sans erreur, mais aucune interprétation n'est attendue.

- `Fsm`, `FsmHeader`, `FsmRule` — machines à états (`fsm synchronous on clk ...`). **Jamais implémenté.**

Hors scope du MVP (éventuellement plus tard)

Ces nœuds sont reconnus par le parser mais non interprétés pour cette itération. Les détecter et lever une `UnsupportedOperationException` avec un message clair suffit.

- `ModuleInstance` — appel d'un sous-module (`ET(a, b : c)` utilisé depuis un autre module).
- `MemoryPoint` — bascules (`:=` avec `on clock`). Nécessite une boucle de convergence.
- `TriState` — sorties trois-états : une sortie qui peut valoir 0, 1 ou Z (haute impédance, « déconnectée »). Utilisé pour partager un bus entre plusieurs émetteurs. Syntaxe SHDL : `sortie = expr when enable`.
- `MapNode`, `MapEntry` — tables de vérité littérales via le mot-clé `map`. Sucre syntaxique pour décrire un décodeur / encodeur sans écrire les équations booléennes (p. ex. `"00" -> "0001"`).

